

# CS4800: Designing Sustainable ICT Systems

## Lecture 3: Sustainable Software

September 17, 2026

Lecturer: Przemysław Pawełczak

Authors: Liwia Padowska, Przemysław Pawełczak

CS4800-EWI@tudelft.nl

# What will you learn in this lecture?

**1** Four laws of green software

**2** Green programming and delivery

**3** Measuring energy and carbon

**4** Green agentic AI

# Disclaimer

- Parts 1 and 2 are based on Wilco Burggraaf's material \*
- This lecture goes beyond the Sustainable Software Engineering course

# Why global CO<sub>2</sub> emissions do not motivate action

41.6 billion  
tonnes

the **world's total yearly** emission of CO<sub>2</sub>e, carbon dioxide equivalent

[Global Carbon Project, 2025](#)

46.9 million  
tonnes

one **large bank's** full supply chain **yearly** footprint

[anonymised sustainability report](#)

81 tonnes

**yearly** emissions of an **IT company**, using 1,000 servers

[Burggraaf, DevOps++ talk](#)

PART 1

# The Four Laws of Green Software

# Four principles of Green Software

1

**Minimize the environmental impact of IT resources** across their full lifecycle, from production to disposal

2

**Design and operate IT systems for longevity**, so they can be maintained, supported, and kept useful over time

3

**Reduce and reuse what already exists** before producing anything new

4

**Do not let a fix in one place overshadow a cost elsewhere**

# Law 1 in practice: Cut redundant work

## Before: one network call per event

```
for event in events:  
    http.post("/log", event)  # one HTTP call per event
```

Each call wakes the radio out of low-power state to send one small event, then lets it go idle again. This repeats for every event.

## After: one batched call for the whole group

```
buffer.append(event)  
if len(buffer) >= BATCH_SIZE or timer_elapsed():  
    http.post("/log/batch", buffer)  # one HTTP call for  
    the whole batch  
    buffer.clear()
```

Events are buffered and sent together, so the radio wakes once for the whole batch instead of once per event.

In a controlled study on Android, batching HTTP calls this way cut energy use by about **40%**: **393 J** instead of **643 J** for the same set of calls.

# Law 1 in practice: More ways to minimize negative impact

## Do not repeat work that has already been done

Useful design patterns from the same family:

- **Cache static data:** if a value has not changed, use the stored copy instead of recomputing or re-fetching it.
- **Lazy loading:** only compute or fetch a value the moment it is actually needed, not before.
- **Batch requests:** group many small calls into one (see previous slide).

## Law 2 in practice: Core observation

- A) Rewriting a system from scratch: paying development cost for a second time
- B) Repairing: it does not, as that cost was already paid once

# Law 3 in practice

## The best product is the one you already have

- Reuse a library instead of building new
- Extending device life by half could avoid half of its total emissions

**Example:** By keeping lab laptops in service **six** years instead of **three**, a university avoids buying and manufacturing twice as many replacement devices.

# Law 4 in practice

## A local win can be a global loss

Fixing one part of a system can quietly push the same cost onto another part.

Always check the whole chain before calling something an improvement.

**Example:** Switching from **server-side** to **client-side rendering** lowers the server load but pushes the same rendering work onto every visitor's device. The total energy use can rise even as server costs fall.

# Law 4: the greenest programming languages

| Language                | Class           | Energy per benchmark run               |
|-------------------------|-----------------|----------------------------------------|
| C                       | Compiled        | 57 J, lowest energy measured           |
| Rust                    | Compiled        | 59 J                                   |
| C++                     | Compiled        | 77 J                                   |
| Java                    | Virtual machine | 114 J                                  |
| Compiled average        | many languages  | about 120 J                            |
| Virtual machine average | many languages  | about 576 J, about 5 times compiled    |
| Ruby                    | Interpreted     | 4,045 J                                |
| Python                  | Interpreted     | 4,390 J                                |
| Interpreted average     | many languages  | about 2,365 J, about 20 times compiled |

# Which language for which task

## A large inner loop

Processing millions of rows, or a training loop that runs constantly.

**Compiled languages win here. C, Rust: about 20 times less energy than the interpreted average.**

## A low traffic internal tool

An admin dashboard used by ten people a week. Engineering time to build, review, and maintain it matters more than runtime energy.

**Scripting languages needed about half the development time and half the code length of C for the same task.**

**A language like Python is often the better default.**

## A data or ML pipeline

Training a model with NumPy in Python. The interpreter barely runs, since the heavy loop underneath is compiled C, C++, or CUDA code.

**A 2024 study: NumPy's matrix multiplication ran 84 times faster and used 44 times less energy than an equivalent hand-written C++ loop.**

# Rating software is still hard

## We cannot yet pick the greenest library

Physical goods have clear life cycles and eco labels. Software is intangible and runs on millions of machines at once, so measuring and assigning its impact is hard.

### Some guidelines to follow:

- Code that does a job in fewer processor cycles usually also uses less energy
- A region whose power comes mostly from wind, solar or water is cleaner to run in
- A project many people maintain has likely already had most of its waste fixed

PART 2

# From CO<sub>2</sub> to Action

# Act on what you can control

Let the CO<sub>2</sub> report of your infrastructure confirm your interventions worked. **A global annual CO<sub>2</sub> total is too broad to act on directly.**

Engineers can reduce emissions **today** by shutting down idle machines, right-sizing resources, or reducing network traffic.

# Four ways to reduce waste

1

## Eliminate idle compute

Avoid leaving machines and containers running with nothing to do

2

## Find the real bottleneck

It often lies in slow disks and databases, not the processor

3

## Move less data

Send smaller messages, and run code closer to the people using it

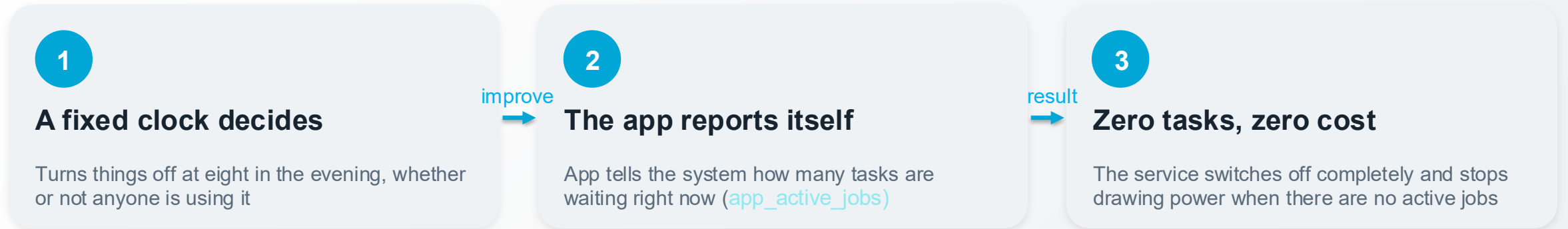
4

## Measure what users feel

Measure response times from real users and count operations behind each user action

# Eliminate idle compute: Turn off what nobody is using

A forgotten test copy can run for weeks unused.



`minReplicaCount: 0` *# allowed to switch off completely*

triggers:

- type: prometheus

query: `sum(app_active_jobs)` *# a number the app reports itself: if there is no work to do, it scales the service to zero*

# Find the real bottleneck: Trace load back to its cause

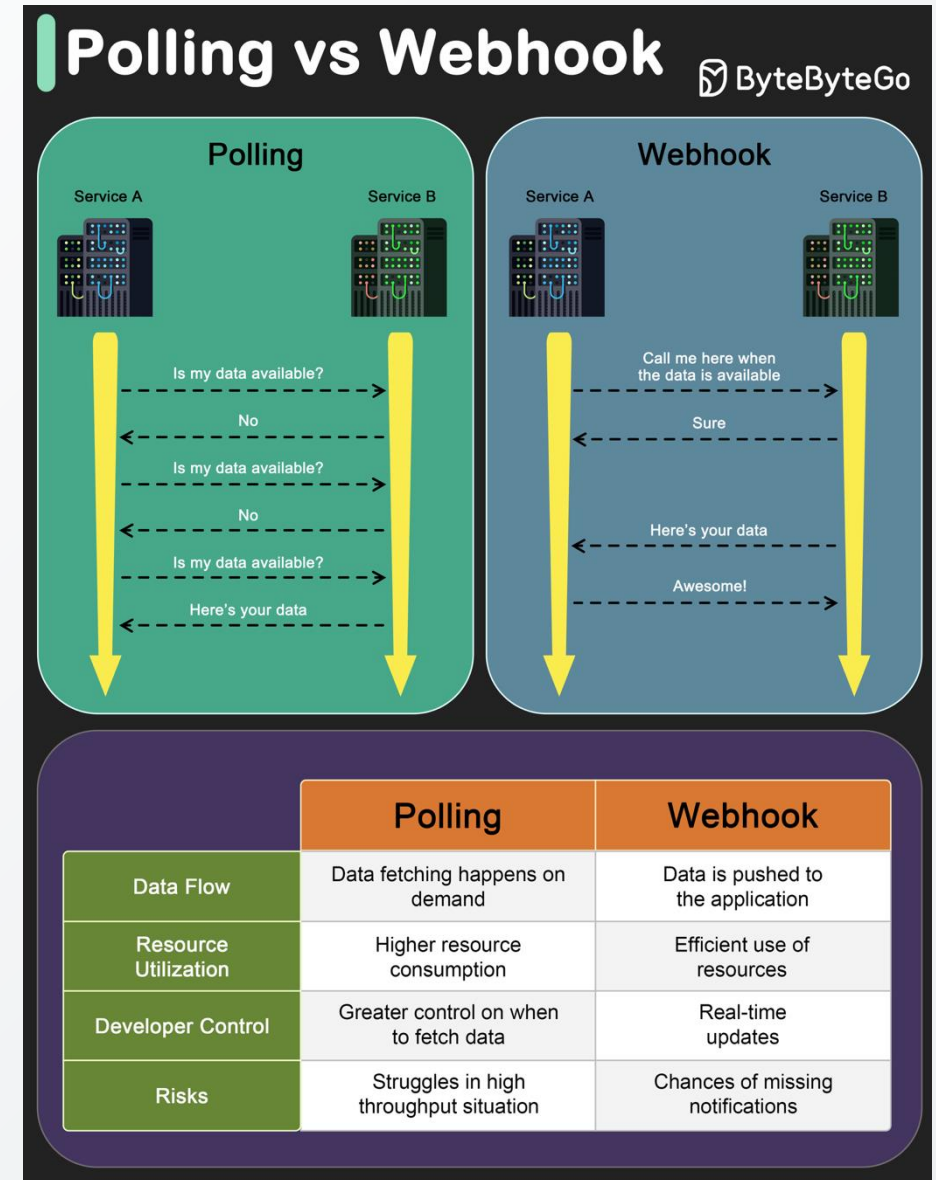
## Is the processor busy for a real reason?

A spike in CPU usage could be:

- 1) a real job matching orders to stock,
- 2) one service asking another "are you done yet?" dozens of times a second — a pattern called polling.

On a monitoring dashboard that averages usage into 5-minute buckets, both show up as the same CPU spike.

**Only (1) is worth paying for with more servers.**



ByteByteGo. (n.d.). Polling vs Webhooks [Diagram]. ByteByteGo.

# Find the real bottleneck: Identify the slow step

A request that takes two seconds might be blamed on the processor when the real delay is the database.

Use small markers placed in the code to time each step:

```
with tracer.start_as_current_span("database_query"):
    order = database.query("SELECT * FROM orders WHERE id = ?", order_id)

with tracer.start_as_current_span("shipping_api_call"):
    status = shipping_api.get_status(order.tracking_id)
```

For example, if the database step took 1.8 seconds and the shipping call took 0.15 seconds then the bottleneck is obvious, and it is not the processor. Adding more servers here would not improve anything.

# Find the real bottleneck: Validate performance early

## Issue caught before release

A test run in the code review stage finds it.  
A few minutes to fix.  
Nothing has been seen by real users yet.  
  
This saves time, energy and budget.

## Issue caught after release

Real users are already affected.  
  
Extra servers may get added, and the inefficient code keeps running and using energy.

# Move less data: Send only the data that gets used

Track which fields in an API response are actually accessed by downstream code.

A field nobody reads is a strong candidate to drop.

```
class TrackedResponse(dict):
    def __getitem__(self, key):
        telemetry.increment(f"field_used.{key}")
        return super().__getitem__(key)

    def get(self, key, default=None):
        telemetry.increment(f"field_used.{key}")
        return super().get(key, default)
```

For example, after running for a while:

- `field_used.email` was read 50,000 times,
- `field_used.middle_name` was read 0 times.

So - drop `middle_name`

# Measure what users feel: Run it in the right place

## Not everything belongs in the cloud

Where response time, data location, or sustainability matter, running code close to the user, on the edge or on their own device, can be the better.

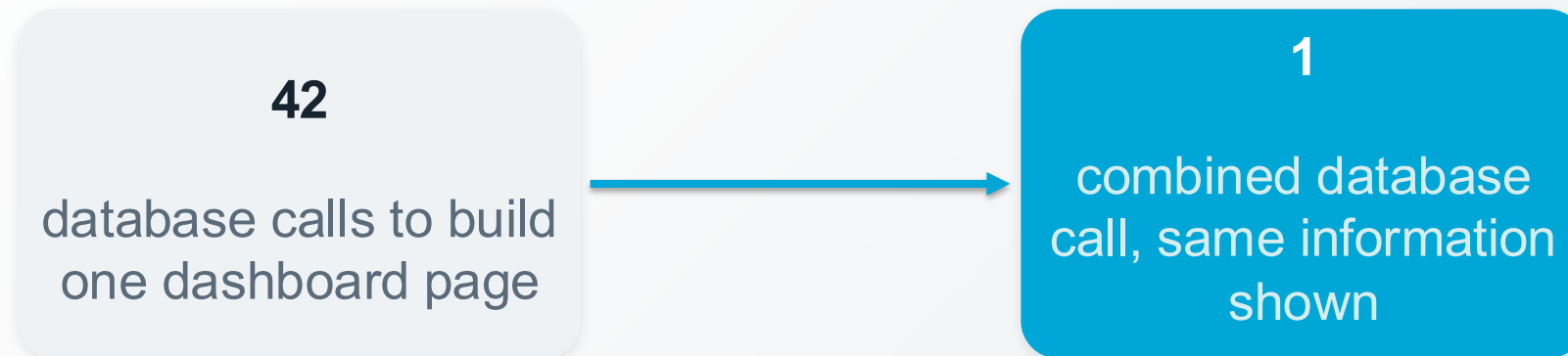
## Bypass long-distance WAN data transmission

Transmitting data over the Internet consumes **~0.015–0.06 kWh/GB** across core routers and radio access networks. Local or edge execution reduces network data transfer by **up to 70%–90%**.

## Lower round-trip latency to accelerate idle sleep

A phone or laptop's Wi-Fi/5G chip stays fully awake at high power while waiting for replies. Cutting response times from ~80 ms to <10 ms lets the wireless chip return to low-power sleep mode immediately instead of draining battery while waiting.

# Measure what users feel: Count the work behind one click



# Measure what users feel: Fast and green *usually* agree

## Four small habits that improve both

- **Start a slow task and move on, instead of waiting idle for it**
- **Combine many small requests into one (batch network calls)**

- **Keep a copy of something expensive nearby, reuse it**

Serving payloads from nearby caches avoids repetitive round-trip bus traversals and eliminates redundant database compute.

- **Calculate a result ahead of time, before anyone asks**

Static rendering and precomputation remove dynamic overhead from critical user paths and enable temporal load shifting to cleaner grid hours.

## Moving data often costs the most energy

Reading an operand from off-chip main memory consumes **1,000× to 10,000×** more dynamic energy than executing a 32-bit addition inside the processor core.

---

### ONE KEY CAVEAT: EMBODIED CARBON

If an optimization requires buying new hardware, it causes a net carbon loss. Remember from our previous lecture - manufacturing already accounts for 50%–80% of a device's lifetime footprint.

# Example of waste removal

## Before: sorts everything, recomputes always

```
def get_info(orders):  
    ordered = sorted(orders, key=lambda o: o.date)  
    active = [o for o in ordered if o.active]  
    groups = group_by(active, lambda o: o.region)  
    return {r: summarize(os) for r, os in groups.items()}
```

Every order is sorted, even the ones thrown away next line.

The order summary runs fresh every time, even with no new data.

## After: filters first, remembers results

```
def get_info(orders, cache):  
    active = [o for o in orders if o.active]  
    active.sort(key=lambda o: o.date)  
    groups = group_by(active, lambda o: o.region)  
    return {r: cache.get_or_compute(r, summarize, os)  
            for r, os in groups.items()}
```

**Early filtering shrinks sorting input:** Filtering first leaves only  $K$  active items. Sorting complexity drops significantly, cutting comparison and memory-swap energy by **up to 70%–85%**.

**Memoization eliminates redundant CPU execution** `get_or_compute()` resolves identical queries in  **$O(1)$**  lookup time, cutting computation energy by **>90%** on cache hits.

**In-place sort avoids secondary list allocation** `active.sort()` mutates the list in place without allocating auxiliary buffers, maximizing L1/L2 cache locality and suppressing garbage collection sweeps.

PART 3

# Measuring Energy and Carbon

# Two kinds of efficiency

## Energy efficiency

How much energy running the application itself uses.

Granular: it can be measured down to one function, one request, or one line of code.

## Carbon efficiency

The environmental cost across the system's whole lifecycle, including manufacturing the hardware it runs on.

Broader, and much harder to change on any single day.

# Measuring: energy and power

**Power** is the instantaneous rate of doing work (measured in **Watts**, 1 W = 1 J/s).

**Energy** is the total physical work performed over an elapsed time (measured in **Joules** or **kilowatt-hours**).

## Operational energy (E)

$$E = P_{avg} \times \Delta t$$

**E** Total energy consumed across active components (Joules, **J**, or **kWh**)

**P<sub>avg</sub>** Average power draw across CPU, memory, and board rails (Watts, **W**)

**Δt** Workload execution runtime / elapsed duration (seconds, **s**)

## Energy-delay product (EDP)

$$EDP = E \times t = P_{avg} \times t^2$$

**E** Total operational energy spent completing the workload (Joules, **J**)

**t** Execution delay / completion latency (seconds, **s**)

**t<sup>2</sup>** Quadratic penalty factor that prevents trading away latency for minimal energy

# Measuring at the wall: the watt meter

The simplest way to measure power: plug the whole system into a meter and read the number.



A plug in watt meter

## How it works

The device under test plugs into the meter, and the meter plugs into the wall. It averages voltage times current over time and shows the result live, in watts.

Most accurate reading of the whole system's real draw, but zero granularity: it cannot tell you which process, or which line of code, is responsible.

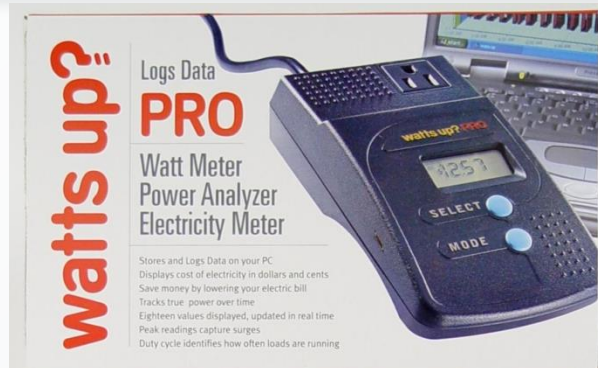
# Measuring at the wall: the watt meter

The same whole system idea as the watt meter, built either for serious logging or for a smart home.

1

## Watts Up? Pro

A logging watt meter used in real published software energy research. Connects over USB or serial and records a full time series automatically, not just one live number.



2

## Wi-Fi smart plug

Consumer plugs with a built in metering chip. Readable in code over their own API, or an open source library, a cheap way to build a DIY logging setup for a home lab.



# Inside a Monsoon high voltage power monitor

A commercial device that connects in place of a mobile device's own battery, for lab grade accuracy.



Monsoon HVPM unit, external view

- 1 Current sense resistor: sits in the device's power line. Its tiny voltage drop reveals the current flowing right now.
- 2 Fast analog to digital converter: samples voltage and current thousands of times a second, far faster than software could poll.
- 3 USB link to a PC: streams every sample to a PC running Monsoon's software, for live viewing, logging, and analysis.

# What a real power trace looks like

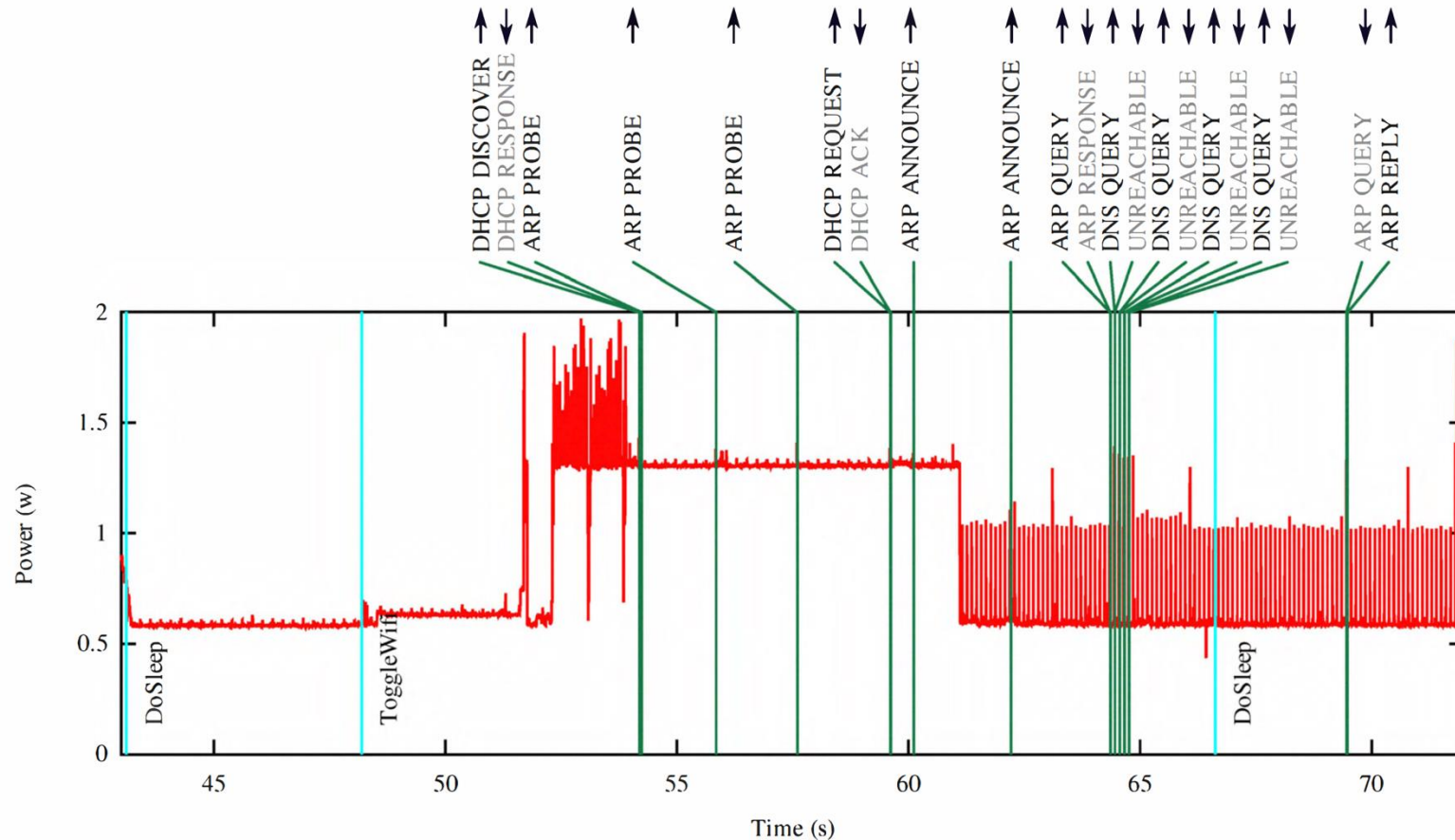
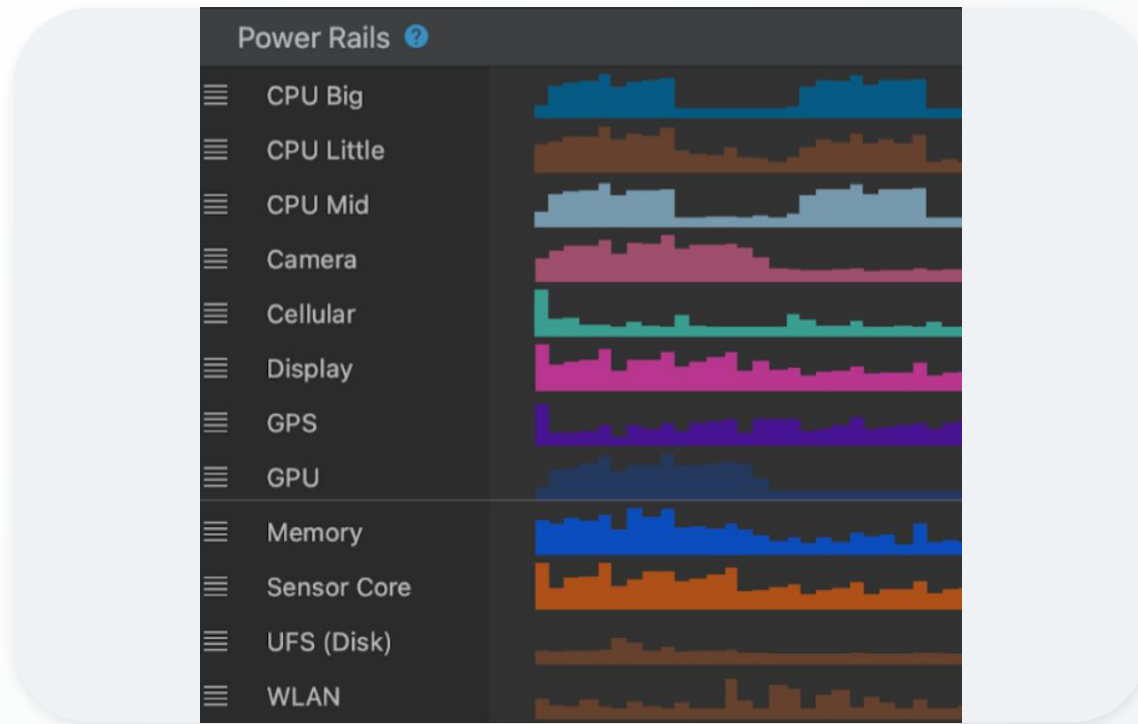


Figure 9. An extract from the energy trace of connecting to a WiFi network and obtaining an IP address, annotated with each IP packet sent and received.

# Profiling on device: Android Studio Power Profiler

Built into the IDE. Live view while debugging an app on a connected phone or a virtual device.



Power Rails panel, Android Studio Profiler

## How students use it

- 1 Run the app under debug, on a connected device or an emulator
- 2 Open the Profiler tab, then the Power Rails panel
- 3 Trigger the action you care about: opening a screen, a location fix, a network call
- 4 Read which rail spiked: the CPU, the GPU, the display

# Reading energy straight from the chip: RAPL

**RAPL**, Running Average Power Limit, is a real energy counter built into most Intel and AMD chips.



The kernel's intel\_rapl driver does this for you at `/sys/class/powercap/intel-rapl:0/energy_uj`. Underneath, that file is just `MSR_PKG_ENERGY_STATUS` (register 0x611), read with the x86 RDMSR instruction and scaled by `MSR_RAPL_POWER_UNIT` (0x606):

```
MSR_RAPL_POWER_UNIT = 0x606    # energy scale factor lives here
MSR_PKG_ENERGY_STATUS = 0x611   # package energy counter (RDMSR, root only)
def rdmsr(cpu, reg):
    fd = os.open(f"/dev/cpu/{cpu}/msr", os.O_RDONLY)
    os.lseek(fd, reg, os.SEEK_SET)
    return struct.unpack("Q", os.read(fd, 8))[0]
energy_unit = 0.5 ** ((rdmsr(0, MSR_RAPL_POWER_UNIT) >> 8) & 0x1F) # joules / tick
raw = rdmsr(0, MSR_PKG_ENERGY_STATUS) & 0xFFFFFFFF # 32-bit counter, wraps ~60s
joules = raw * energy_unit
```

# Sharing that energy between programs: Scaphandre

Scaphandre is a tool that reads the chip counter above, then splits it fairly between running processes.



```
// scaphandre/src/sensors/mod.rs (simplified)
pub fn get_process_power_consumption_microwatts(&self, pid: Pid) -> Option<Record> {
    let cpu_pct = self.get_process_cpu_usage_percentage(pid)?;
    let machine_delta = self.get_records_diff_power_microwatts()?; // from RAPL, above
    let share = machine_delta.value.parse::<f64>().unwrap()
        * cpu_pct.value.parse::<f64>().unwrap() / 100.0;
    Some(Record::new(machine_delta.timestamp, share.to_string(), Unit::MicroWatt))
}
```

# Beyond RAPL: sensors built into the system

| Tool            | Where it reads from                                                                   | Example                                                       |
|-----------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------|
| nvidia-smi      | An NVIDIA GPU's own onboard power sensor                                              | <code>nvidia-smi --query-gpu=power.draw --format=csv</code>   |
| ipmitool        | A server's baseboard management controller                                            | <code>ipmitool dcmi power reading</code>                      |
| powermetrics    | Apple Silicon's own on die power sensors                                              | <code>sudo powermetrics --samplers cpu_power,gpu_power</code> |
| INA219 / INA226 | A current sense chip wired onto a board's own power rail, common in DIY research rigs | Read over I2C, logged from a Raspberry Pi power rig           |

nvidia-smi: [NVIDIA developer docs](#). ipmitool: [ipmitool.org](#). INA219 example: [Stokke, Stensland, Griwodz, & Halvorsen \(2015\), on a Jetson TK1](#).

# Do the two approaches agree?

## Sometimes

- Usually agree: on a dedicated machine, doing mostly processor work
- Often disagree: on a rented cloud machine, or graphics heavy work

## Always state what was measured

A real chip counter and an estimate can both be right and still give different numbers, because they cover different parts of the machine.

Five joules from a real chip counter and seven joules from an estimate are not a contradiction if one covers only the processor and the other covers the whole container.

# One shared score: Software Carbon Intensity (SCI)

An ISO standard from the Green Software Foundation (lower is always better)

$$\text{Score} = (\text{Energy used} \times \text{How dirty that energy is} + \text{Cost of building the hardware}) / \text{One user action}$$

**E**

## Energy used

Kilowatt hours, across servers, storage and the network

**I**

## How dirty

Grams of CO2 per kilowatt hour, changes by place and time

**M**

## Hardware cost

The chip's own footprint, spread across its lifetime

**R**

## One user action

Per user, per request, per training run, whatever is compared

# Example of using SCI

## One backend service, one release

**E** = 1,200 kilowatt hours used (the energy)

**I** = 180 grams of CO<sub>2</sub> per kilowatt hour (how dirty that energy is)

**M** = 60,000 grams of CO<sub>2</sub> (the hardware's share of the embodied cost)

**R** = 40,000 requests served (the functional unit)

$$\text{Score} = (E \times I + M) / R = (1,200 \times 180 + 60,000) / 40,000 \approx 6.9 \text{ grams of CO}_2 \text{ per request}$$

PART 4

# AI-Assisted Development

# How AI helps code stay maintainable

## AI lowers the cost of upkeep

Tests, documentation, and small modular refactors keep code adaptable, but they cost engineering time, so they are often the first thing skipped under deadline pressure.

An AI assistant can draft a first version of each: a test before a refactor, a docstring before review, a small change instead of one big rewrite.

## AI scans, a person still decides

An AI assistant can scan code together with time and energy profiler output to point at likely waste, the same nested loops and repeated work that drive up both.

A person still checks the fix: a slow disk or network call can fool a time profiler alone, so a human confirms it with tests before it ships.

# What makes an AI a good coding partner

Dong, Shi, Sampath & Macvean collected 91 real rule files that engineering teams write to steer their AI coding assistants, then analysed what engineers actually ask for.

**Follow standards and workflows:** match the team's existing conventions, don't invent new ones

**Keep quality and reliability high:** don't trade correctness for speed

**Solve problems incrementally:** check the context first, then make the smallest change that works

**Communicate, don't guess:** ask for clarification when a request is unclear

These same four expectations make code durable and efficient, not just correct. Ask an assistant to weigh trade offs, and energy and carbon can count among them.

# The rules, in their own words

Three real examples pulled from those 91 files (Dong, Shi, Sampath & Macvean, 2026).

*# AGENTS.md, instructions for the assistant*

Make the smallest possible change, then run the tests.

Write code for someone else to read and maintain later.

If a request is unclear, ask, do not guess.

These same rules make code durable and efficient. Ask an assistant to weigh trade offs, and you can tell it energy and carbon count among them.

# Using AI to make software greener: examples

## Spot the N+1 query

*"Here's my slow endpoint. Which of these database calls could be batched into one?"*

## Suggest a leaner algorithm

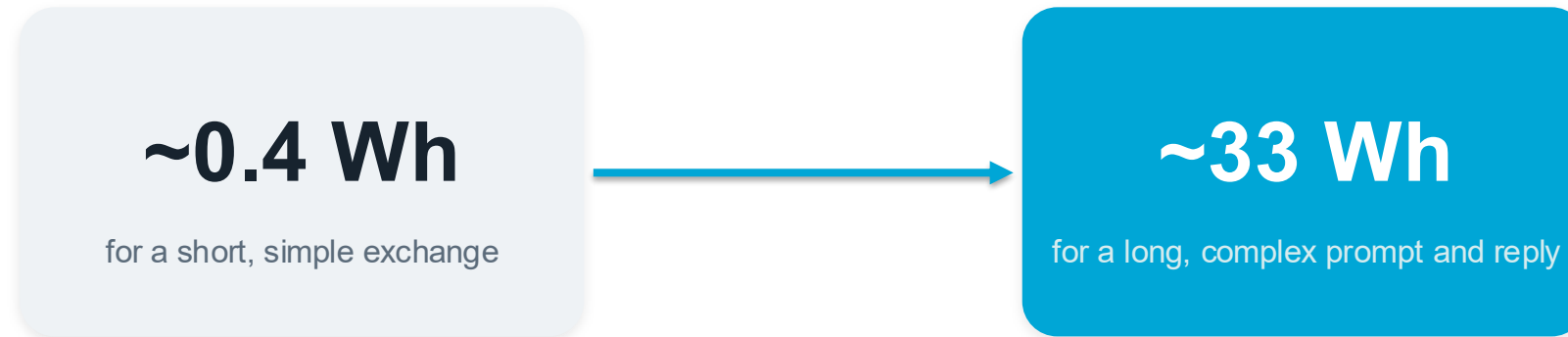
*"Is there a more efficient way to do this than scanning the full list every time?"*

## Write the missing test first

*"Before we refactor this for speed, write a test that pins down its current behaviour."*

# Vibe coding: what does generating it cost

Vibe coding means describing intent in plain language and accepting an AI's output, often unread.



Length and complexity drive the cost, so trying again and again for a better answer adds up fast.

Choosing the smallest model that can do the job still matters more than any other single choice.

# Vibe coding: correct is not the same as efficient

## Correct code can be slow forever

Solovyeva, Weidmann & Castor (2025) asked GitHub Copilot, GPT-4o, and OpenAI o1-mini to solve 53 hard LeetCode problems in Python, Java, and C++, then measured the energy their generated code used against canonical human solutions.

Sorting, graph, and greedy algorithms showed the biggest gap. In C++, the AI generated solutions used 127% to 232% of the human baseline's energy, worst for GitHub Copilot. Python code came close to the human baseline, sometimes beating it.

Not always worse: one o1-mini solution used a technically worse  $O(n^2)$  approach but ran leaner than the human  $O(n)$  version, because it avoided expensive modulus operations inside the loop. Complexity on paper is not the same as energy in practice.

**If nobody reads the output, an inefficient but correct answer ships and burns energy on every run for the life of the system.**

# The responsible stance: measure it

## Prompting for efficiency is not reliable

AI can speed up the Detect step, and can be asked directly to write for efficiency, but asking does not reliably work: Cappendijk et al. (2025) found the same efficiency focused prompt cut energy by 60% on one problem, and increased it by 465% on another, depending on the model.

The responsible stance is to measure the actual result rather than trust the prompt: run the same profiler discipline used everywhere else in this lecture, **and only keep an AI suggested change if it verifiably reduces energy on the real workload.**

This keeps AI assisted development inside the same Software Carbon Intensity boundary as the rest of the system. It is one more thing to measure, not an exception.

# Two lenses on agentic AI

## Lens A: it can make code more maintainable

- Copilot generated code matched human energy use by default, and using it while explicitly prompting for efficiency measurably cut energy, with no drop in correctness
- Counterpoint: human optimised code still beat all three tested AI tools on efficiency in most challenges tried

## Lens B: running the model has a real cost

- Generating text used about 15 times more energy than classifying it, per 1,000 uses; generating an image used about 60 times more than generating text
- An **agentic** coding run can spend up to 1,000 times more tokens than one chat reply, and extra token spend stops improving accuracy well before it stops climbing

# Finding the sweet spot

## What the AI part comes down to

AI can find waste, suggest greener code, and draft routine maintenance work, but every use has its own energy cost and still needs a person to check the result.

- Match effort to the task: a lightweight assistant for a small edit, a budgeted agentic run only when the value clearly justifies it
- Generating code is not free: prompt length, retries, and model size all add to the cost
- Correct is not the same as efficient: measure the result, the same discipline used everywhere else in this lecture

The pay once, save repeatedly idea, that a maintainable AI written codebase keeps saving energy for years, is a reasonable hypothesis to raise with students. It is not yet a settled result in the literature.

# Recap

## Four laws, with real patterns

Use less and cleaner, keep it alive, reuse what exists, see the whole system

## Green programming

The same skill as performance work, judged by energy instead of speed

## Know what each tool measures

A wall socket meter, a chip counter, and a cluster level estimate each cover a different slice

## Fix waste first

A concrete engineering fix is what people can act on; CO2 confirms it worked

## A smaller bill is not always less carbon

Packing work onto fewer machines proved that

## AI cuts both ways

It can make a codebase greener, but running it has a real energy cost of its own

# Take Home Exercise: Profile it yourself

## In groups

You are given a small script that does two things: a tight numeric loop, and a batch of small repeated calls to a mock database. Profile it with two different tools from this part, then compare.

- Pick one software profiler (CodeCarbon, or your machine's RAPL reading) and one hardware or socket level reading (a lab watt meter if available, otherwise nvidia-smi or powermetrics on your own laptop)
- Run the unmodified script under both, and record what each one reports
- Apply the batching fix from Law 1 to the repeated calls, and profile again with both tools
- Write down: did the two tools agree on the size of the improvement? If not, what does each one actually cover that the other does not?

Deliverable: one paragraph naming which tool you trust more for this specific workload, and why, referencing what each tool covers, from the slides earlier in this part.

# Lecture Feedback



# Exam(ple) questions

1. Which statement about reading energy through the `MSR_PKG_ENERGY_STATUS` register is false?
  - a. ~~The register is read with the x86 RDMSR instruction, which requires root access~~
  - b. ~~The raw counter is 32 bits wide and wraps back to zero roughly every 60 seconds under heavy load~~
  - c. ~~The scale factor that converts raw ticks into joules is read from a separate register, `MSR_RAPL_POWER_UNIT`~~
  - d. ~~Reading the counter once, right when your code finishes running, tells you exactly how many joules that run used~~
2. Which statement about the batching example (buffering log events instead of sending one HTTP call per event) is false?
  - a. ~~The batched code sends the whole buffer in a single HTTP call once `BATCH_SIZE` is reached or a timer elapses, whichever comes first~~
  - b. ~~In the unbatched version, every single event wakes the radio out of its low power state, even if several events arrive close together~~
  - c. ~~The savings come from letting the radio return to a low power idle state between transmissions instead of waking it for every event~~
  - d. ~~Batching removes the network calls entirely and replaces them with local storage, so no data reaches the server until the next deploy~~